



SwasthyaSetu

Phase III — Functional Requirements & Technology Specification

Submitted By: **Pookiedevs**

This document builds on the Phase II project proposal and translates the SwasthyaSetu vision into a concrete set of functional requirements, additional features, and the technology stack the team will use to implement them in Phase III.

Functional Requirements

The following are the core, must-have capabilities SwasthyaSetu must deliver to fulfil its purpose as a first-response health companion.

User Registration & Authentication:

Patients can register and log in using phone-number-based OTP verification. Doctors and medical practitioners go through a separate registration flow that requires uploading credentials (medical license/degree) for admin review before their account is activated.

Emergency SOS Request:

A one-tap SOS button lets a patient instantly request help. The system attempts to connect them with the nearest available volunteer doctor and, if no doctor responds within a set time window, escalates by notifying the patient's saved emergency contacts.

AI-Assisted Symptom Triage:

A voice- or text-driven symptom checker, powered by an on-device MedGemma-based model, asks the user guided questions and classifies the situation as Mild, Moderate, or Emergency, recommending an appropriate next step (self-care, facility visit, or immediate doctor call).

Doctor Matchmaking & Queuing:

A matchmaking algorithm routes incoming consultation requests to available, verified doctors based on specialty, current availability, and language preference, and queues requests when no doctor is immediately free.

Live Video & Voice Consultation:

Patients and doctors connect through real-time WebRTC video calls, with automatic fallback to voice-only mode when bandwidth is too limited for video.

First-Aid & Medical Guidance Library:

A searchable, categorized library of first-aid instructions and basic medical guidance, available in Nepali with optional voice narration, and fully accessible offline.

Nearby Health Facility Locator:

A map-based tool showing the nearest health posts, hospitals, and pharmacies, including operating hours and contact details, with facility data cached for offline lookup.

Offline Mode & Background Sync:



Core content — the first-aid library, cached facility data, and the on-device AI model — remains usable without an internet connection. Consultation notes and triage results queued while offline sync automatically once connectivity is restored.

Nepali Voice Interface:

Speech-to-text input and text-to-speech output in Nepali allow users with low literacy or typing difficulty to interact with the app entirely by voice.

Consultation History & Health Records:

Patients can review past triage results, consultation notes, and doctor recommendations. Doctors can view a patient's relevant history during a consultation, subject to the patient's consent.

Doctor Verification & Credentialing:

An admin-facing workflow for reviewing uploaded medical credentials, approving or rejecting doctor applications, and managing active volunteer status.

Notifications & Alerts:

Push notifications (with SMS fallback for feature-phone users) inform patients and doctors of consultation status, queue position, and emergency escalation updates.

Admin Dashboard:

A management console for platform administrators covering doctor approvals, first-aid content management, usage analytics, and overall system health monitoring.

Privacy & Consent Management:

Explicit, revocable consent flows govern how health data is stored and shared, and all analytics collected from usage patterns are aggregated and anonymized.

Additional Features

Beyond the core requirements, the following features extend SwasthyaSetu's reach and are planned as stretch goals for Phase III, resources permitting.

Community Health Worker Mode:

A dedicated mode for local health volunteers to log home visits, record basic patient observations, and escalate cases directly to a volunteer doctor on the platform.

SMS / USSD Fallback:

A lightweight interaction path over SMS or USSD that gives users without smartphones access to basic triage guidance and the nearest facility's contact details.

Medicine & Follow-up Reminders:

Scheduled push or SMS reminders to help patients keep to medication schedules and follow-up consultation dates set by a doctor.

Emergency Contact Auto-Dial:

Alongside the SOS request, the app can simultaneously place a call or send an alert to the user's pre-configured emergency contacts.

Doctor Rating & Feedback:



After each consultation, patients can leave a rating and short feedback, helping build doctor credibility and giving the admin team a quality signal.

Family Health Profiles:

A single account can manage linked health profiles for dependents, such as children or elderly parents, so a caregiver can act on their behalf during an emergency.

Community Health Awareness Feed:

A curated feed of seasonal health alerts (e.g. monsoon-related illness, dengue outbreaks) and general wellness tips sourced from verified public-health content.

Ambulance / Transport Request:

Where locally available, a button to request ambulance or transport services directly from an active emergency session.

Tools & Technologies

The Phase III technology stack builds on the team's chosen frontend and backend frameworks — Next.js and Python (FastAPI) — alongside supporting tools recommended for this stack. Items marked as team-specified reflect explicit decisions already made; the rest are suggested choices that can be adjusted in the team's technical design meeting.

Frontend

- Next.js (React framework, used as a PWA) — team-specified
- TypeScript for type-safe application code
- Tailwind CSS for styling
- shadcn/ui for pre-built accessible UI components
- Zustand for lightweight client-side state management

Backend

- Python — team-specified
- FastAPI as the web framework — team-specified
- Uvicorn/Gunicorn as the ASGI server
- SQLAlchemy (or SQLModel) as the ORM, with Alembic for database migrations
- Pydantic for request/response data validation (built into FastAPI)
- Celery for background jobs and async task processing
- PostgreSQL as the primary relational database
- Redis for caching, session storage, and Celery's task queue

Real-Time & Communication

- WebSockets (native to FastAPI) for live status updates and messaging
- WebRTC with a TURN/STUN server for video and voice consultations
- Twilio (or equivalent) for OTP and SMS/USSD fallback delivery



- Firebase Cloud Messaging for push notifications

AI / ML

- MedGemma as the base model for symptom triage
- Whisper.cpp for on-device Nepali speech recognition
- A Nepali text-to-speech model for voice output

DevOps & Infrastructure

- Docker and Docker Compose for containerized environments
- GitHub Actions for CI/CD pipelines
- Cloud hosting on AWS, GCP, or Azure (final provider to be decided by the team)